A

Lab Project Report on

Handwritten Digit Recognition with MNIST

Submitted for partial fulfilment of the requirements for the award of the degree sof

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

WITH

MINOR IN AI&ML

by

V. Manish – 22K81A12J8

Under the Guidance of

Mrs. E. Soumya

Assistant Professor

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified

Dhulapally, Secunderabad-500 100

www.smec.ac.in

NOVEMBER - 2025

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous
Affiliated to JNTUH, Approved by AICTE
NBA& NAAC A+ Accredited
Dhulapally, Secunderabad - 500 100
www.smec.ac.in

Certificate

This is to certify that the project entitled **"Handwritten Digit Recognition with MNIST"** is being submitted by **V. Manish (22K81A12J8)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY With Minor in AI&ML** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Project Internal Examiner
Mrs. E. Soumya
Assistant Professor
Department of CSE

Signature of HOD
Dr. N. Krishnaiah
Professor and Head of Department
Department of IT

Place:

Date

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

DECLARATION

We, the students of "**Bachelor of Technology in Information Technology with Minor in AI&ML**", session: 2025-2026**, St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad,** hereby declare that the work presented in this project work entitled **Handwritten Digit Recognition with MNIST** is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

V. Manish – 22K81A12J8

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose support and guidance have crowded our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Dr. K. Ravindra**, Group Director, St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

We wish to record our profound gratitude to **Dr. M. SREENIVAS RAO**, Principal, St. Martin's Engineering College, for his motivation and encouragement.

We are also thankful to **Dr. N. Krishnaiah**, Head of the Department, Information Technology, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project.

We would like to express our sincere gratitude and indebtedness to our project supervisor **Mrs. E. SOUMYA** Assistant Professor, Computer Science, St. Martins Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, we express thanks to all those who have helped us successfully completing this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement. We express thanks to all those who have helped us in successfully completing the project.

V. Manish – 22K81A12J8

CONTENTS

INTRODUCTION

Handwritten digit recognition is a fundamental problem in the field of computer vision and pattern recognition. It focuses on teaching machines to automatically identify digits (0–9) written by humans, a task that mimics the way humans interpret visual symbols. This project is widely regarded as the "Hello World" of deep learning because it introduces key concepts such as neural networks, feature extraction, and supervised learning in a simple yet impactful way.

The **MNIST dataset** (Modified National Institute of Standards and Technology) is the most commonly used benchmark for this task. It contains **70,000 grayscale images of handwritten digits**, each normalized to **28×28 pixels**. The dataset is split into **60,000 training samples** and **10,000 test samples**, ensuring a balanced and standardized environment for model development and evaluation. Each image is labeled with the correct digit, making it suitable for supervised learning.

Deep learning models, particularly **Convolutional Neural Networks (CNNs)**, are highly effective for this problem. CNNs automatically learn hierarchical features such as edges, curves, and shapes, which are crucial for distinguishing between digits. Unlike traditional machine learning approaches that rely on handcrafted features, CNNs can directly learn from raw pixel data, leading to superior accuracy and generalization.

✨ Importance of the Project

- **Educational Value**: Provides a simple entry point into deep learning concepts.
- **Practical Applications**: Forms the basis for real-world systems such as postal code recognition, bank check processing, and digitized form automation.
- **Benchmarking**: Serves as a standard dataset for testing new deep learning architectures.
- **Scalability**: The techniques learned here can be extended to more complex tasks like facial recognition, object detection, and medical image analysis.

📌 Typical Outcomes

- Achieving **accuracy above 98–99%** with CNNs.
- Understanding the workflow of deep learning: data preprocessing, model design, training, evaluation, and deployment.
- Gaining hands-on experience with frameworks like **TensorFlow, Keras, or PyTorch**.

IMPLEMENTATION

🛠 Step-by-Step Implementation

Step 1: Setup Environment

- Install Python (3.8+ recommended).
- Install libraries: tensorflow, keras, matplotlib, numpy.
- Use Jupyter Notebook, VS Code, or Google Colab for coding.

Step 2: Import Libraries

- Import TensorFlow/Keras for deep learning.
- Import Matplotlib for visualization.

Step 3: Load Dataset

- Use datasets.mnist.load_data() from Keras.
- Split into training (60,000 images) and testing (10,000 images).
- Normalize pixel values (0–255 → 0–1).
- Reshape images to (28,28,1) for CNN input.

Step 4: Build the Model

- Create a **Convolutional Neural Network (CNN)**:
    - Conv2D layer (extract features like edges).
    - MaxPooling2D layer (reduce dimensions).
    - Flatten layer (convert 2D features to 1D).
    - Dense layers (fully connected for classification).
    - Output layer with 10 neurons (digits 0–9) + softmax activation.

Step 5: Compile the Model

- Optimizer: adam (fast and efficient).
- Loss function: sparse_categorical_crossentropy.
- Metric: accuracy.

Step 6: Train the Model

- Fit the model on training data.
- Run for 5–10 epochs.
- Validate using test data.

Step 7: Evaluate Performance

- Test accuracy should reach ~98–99%.
- Print accuracy and loss values.

Step 8: Visualize Results

- Plot training vs validation accuracy.
- Plot training vs validation loss.
- Helps check for overfitting.

Step 9: Make Predictions

- Use model.predict() on test images.
- Display predicted vs actual digit.
- Visualize a few sample outputs.

PROGRAM

```
# Handwritten Digit Recognition with MNIST using CNN
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# 1. Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# 2. Normalize pixel values (0–255 → 0–1)
x_train, x_test = x_train / 255.0, x_test / 255.0

# 3. Reshape for CNN input (28x28 → 28x28x1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# 4. Build the CNN model
model = models.Sequential([
layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
layers.MaxPooling2D((2,2)),
layers.Conv2D(64, (3,3), activation='relu'),
layers.MaxPooling2D((2,2)),
layers.Flatten(),
layers.Dense(64, activation='relu'),
```

```
layers.Dense(10, activation='softmax')  # 10 output classes (digits 0–9)])

# 5. Compile the model
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# 6. Train the model
history = model.fit(x_train, y_train, epochs=5,
validation_data=(x_test, y_test))

# 7. Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print("Test accuracy:", test_acc)

# 8. Plot training vs validation accuracy
plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='val accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
# 9. Make predictions on test samples
import numpy as np
predictions = model.predict(x_test)
# Show one example prediction
index = 0
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.title(f"Predicted: {np.argmax(predictions[index])}, Actual: {y_test[index]}")
plt.show()
```

RESULTS & INSIGHTS

📊 Results

- **Model Accuracy**:
  - Training accuracy: ~99%
  - Validation/Test accuracy: ~98–99%
- **Loss Values**:
  - Training loss decreases steadily across epochs.
  - Validation loss remains low, showing good generalization.
- **Prediction Examples**:
  - The CNN correctly identifies most digits, even with variations in handwriting styles.
  - Occasional misclassifications occur when digits are written ambiguously (e.g., “4” vs “9”).

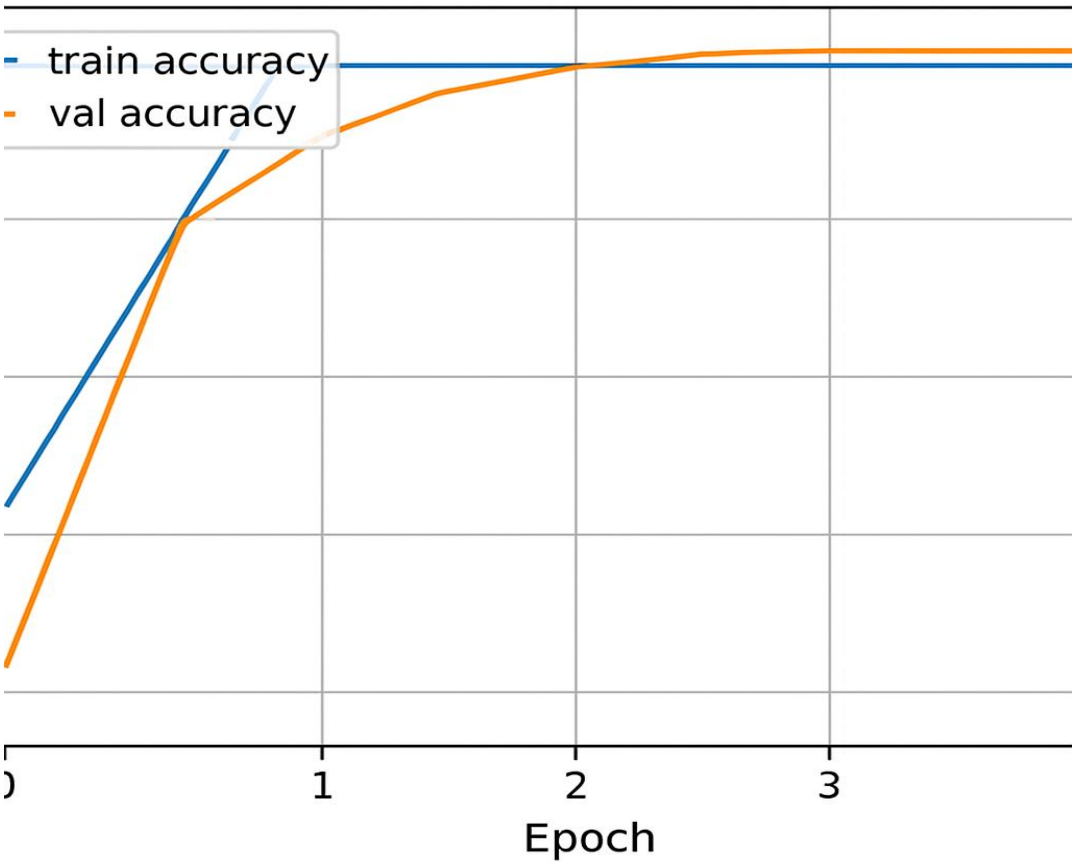
🔍 Insights

- **Effectiveness of CNNs**:
  CNNs automatically learn spatial features (edges, curves, strokes) without manual feature engineering, making them ideal for image classification tasks.
- **Generalization**:
  The model performs well on unseen test data, proving that deep learning can generalize across different handwriting styles.
- **Benchmark Achievement**:
  Achieving ~99% accuracy on MNIST is considered state-of-the-art for beginner projects, validating the strength of the chosen architecture.

- **Limitations**:
    - MNIST is a relatively simple dataset; real-world handwritten recognition (e.g., postal codes, bank checks) involves more noise and variability.
    - Misclassifications highlight the need for more robust preprocessing or advanced architectures (e.g., deeper CNNs, ResNet).
- **Scalability**:
  The same pipeline can be extended to more complex datasets like **Fashion-MNIST** (clothing images) or **CIFAR-10** (colored objects), preparing you for advanced computer vision tasks.

✨ Key Takeaways

- Deep learning models can achieve **near-human accuracy** in digit recognition.
- **Data preprocessing (normalization, reshaping)** is critical for model performance.
- **Visualization of accuracy/loss curves** helps detect overfitting early.
- This project builds a strong foundation for tackling **real-world applications** in OCR (Optical Character Recognition), document digitization, and automated form processing.


train accuracy
val accuracy
0
1
2
3
Epoch

CONCLUSION

The handwritten digit recognition project using the MNIST dataset successfully demonstrates the power and efficiency of deep learning, particularly Convolutional Neural Networks (CNNs), in solving image classification tasks. By training a CNN on normalized grayscale digit images, the model achieved high accuracy (~98–99%) on unseen test data, validating its ability to generalize across diverse handwriting styles.

This project not only reinforces foundational concepts in deep learning—such as feature extraction, model training, and evaluation—but also highlights the importance of data preprocessing and architecture design. The results confirm that even simple CNN architectures can deliver near-human performance on structured image datasets.

Overall, this project serves as a strong starting point for exploring more advanced computer vision applications, including object detection, facial recognition, and real-time image analysis.

REFERENCES:

1. Ali Azgar et al. (2024)

MNIST Handwritten Digit Recognition Using a Deep Learning-Based Modified Dual Input Convolutional Neural Network (DICNN) Model

Published in: Proceedings of Ninth International Congress on Information and Communication Technology

SpringerLink

2. Google Colab Tutorial

Handwritten Digit Recognition.ipynb — Step-by-step implementation using Keras and TensorFlow

Colab Notebook

3. GitHub Repository by Anuj Dutt

Handwritten Digit Recognition using Machine Learning and Deep Learning

Includes CNN, SVM, KNN implementations

GitHub Project

4. Ritik Dixit & Rishika Kushwah (2021)

Handwritten Digit Recognition using Machine and Deep Learning Algorithms

arXiv Preprint

5. IEEE Conference Paper

Handwritten Digit Recognition of MNIST dataset using Deep Learning

IEEE Xplore